

Best Practice for AI Agents Project

Chapter 2

Injecting Private Knowledge without Retraining

Junxian Zhu¹ Yiran Sun² Guanping Dai³

LinkMind Project

¹ Junxian Zhu, LinkMind Project Team, [LandingBJ](#)

² Yiran Sun, LinkMind Project Team, [Xiamen University Malaysia](#)

³ Guanping Dai, LinkMind Project Team, Founder and CEO of [LandingBJ](#)

Contents

About the Contributors	2
Chapter 2. Injecting Private Knowledge without Retraining	3
Learning Objectives	4
2.1 The Knowledge Gap in Enterprise Agents	4
2.2 RAG as an Engineering Pipeline	5
2.3 Source Preparation, Chunking, and Metadata	6
2.4 Retrieval Strategies beyond “Top-K Vector Search”	7
2.5 Grounded Generation and Refusal Behavior	7
2.6 Hands-On Lab: Building an Internal Policy Assistant	8
2.7 Metrics and Evaluation for Private Knowledge Systems	11
2.8 Common Anti-Patterns	12
2.9 Multi-Source Knowledge and Structured Data	12
2.10 Provenance, Citation, and Evidence Presentation	13
2.11 Knowledge Governance over Time	14
2.12 When Retrieval Is Not Enough	14
Chapter Summary	15
Review Questions	15
Further Study: Conflicting Truth and Source Priority	16
Further Study: Review Programs and Benchmark Stewardship	17

About the Contributors

Junxian Zhu

LinkMind Project Team, [LandingBJ](#)

- Focus on the research and development of large model products, covering algorithms and applications
- Rich experience in full-process development and delivery of government and enterprise projects

Yiran Sun

LinkMind Project Team, [Xiamen University Malaysia](#)

- Focus on AI agent engineering practices and applied AI workflows
- Background in computer science, machine learning, and AI-related system research

Guanping Dai

LinkMind Project Team, Founder and CEO of [LandingBJ](#)

- Over 20 years of experience in AI algorithm research and development. Built proprietary deep learning and large-model frameworks
- Extensive experience in banking and telecommunications system architecture. Participated in image analysis projects for the National Automotive Safety Laboratory and telemetry tracking systems for Shenzhou spacecraft recovery capsules
- Former experience at the Chinese Academy of Sciences Software Institute
- Former experience at BEA/Oracle (China)

2

CHAPTER

Injecting Private Knowledge without Retraining

Enterprises rarely struggle with language generation in the abstract. Their deeper struggle is that the model does not live inside the enterprise's changing body of truth. Policies change. Contracts change. Pricing changes. Teams change ownership boundaries. Support articles are revised. Security guidance is amended after incidents. The model may still produce persuasive language, but persuasive language is not the same thing as answerability grounded in current organizational reality.

Private knowledge injection addresses this problem by turning freshness into a systems pipeline rather than a training event. Instead of assuming that model weights must somehow absorb every internal document, the system retrieves relevant evidence at runtime and injects it into the answer path. This move is more than a convenience. It changes economics, governance, and explainability. New documents can become visible without retraining. Sources can be cited. Access boundaries can be expressed through metadata and permissions instead of being irreversibly mixed into a giant statistical memory.

LinkMind's Knowledge Base Server and Embedding Server patterns are useful teaching devices because they expose knowledge injection as a first-class platform concern. The system no longer pretends that "using enterprise data" is a mystical property of the model itself. It makes visible the pipeline of ingestion, chunking, metadata preservation, vectorization, retrieval, filtering, and answer construction. That visibility is the beginning of good engineering.

Learning Objectives

- Explain why private knowledge and pretrained model memory solve different problems.
- Understand the full RAG pipeline from ingestion to grounded answer generation.
- Design chunking, metadata, and retrieval strategies that support enterprise governance.
- Build a policy assistant on LinkMind that can be updated without model retraining.
- Evaluate grounding, freshness, refusal quality, and provenance rather than relying on anecdotal confidence.

2.1 The Knowledge Gap in Enterprise Agents

A large language model contains broad public knowledge and flexible pattern recognition, but enterprises need something more exacting. They need answers that can distinguish an obsolete policy from a current one, a local team practice from an official standard, and a restricted document from a public explanation. In other words, they need not only linguistic competence but institutional grounding.

This is why generic chat quality can be misleading. A model may appear knowledgeable when asked about procurement, travel approval, or security procedures because it can imitate the language of business policy. Yet that imitation can be dangerous. The answer may be plausible while still being non-authoritative, stale, or inconsistent with internal rules. A grounded system must therefore privilege evidence over eloquence whenever the two conflict.

The knowledge gap also has a temporal dimension. Retraining or fine-tuning can improve style or domain familiarity, but it is usually too blunt and too slow to serve as the only mechanism for operational truth. Enterprises change daily. A policy assistant that requires model retraining for every meaningful revision is economically and organizationally misaligned with the speed of the business it is meant to support.

2.2 RAG as an Engineering Pipeline

Retrieval-augmented generation, or RAG, is often described too narrowly as “search plus generation.” A more accurate description is that it is a runtime knowledge pipeline composed of at least six linked stages: source acquisition, normalization, chunking, vectorization, retrieval and filtering, and answer framing. Quality depends on the interaction of all six. A weak document cleaning step can poison chunk boundaries. Poor metadata design can make permission enforcement impossible. Naive retrieval can surface superficially similar but operationally irrelevant passages. An answer prompt that over-encourages helpfulness can still hallucinate even when the right evidence was retrieved.

Thinking in pipeline terms has several benefits. It distributes responsibility across tractable components rather than attributing everything to the mysterious intelligence of the model. It also makes iteration cheaper. If grounding is poor, the team can ask whether the issue is chunk size, metadata, embeddings, filters, or generation instructions instead of repeatedly replacing models in hope rather than diagnosis.

This pipeline view also clarifies where LinkMind fits. LinkMind is not the knowledge itself. It is the mediator that connects knowledge infrastructure to the agent runtime. It can present a consistent retrieval interface upward while allowing the enterprise to evolve embeddings, vector stores, filters, or categories underneath. That separation is essential if several agent surfaces must share one body of truth.

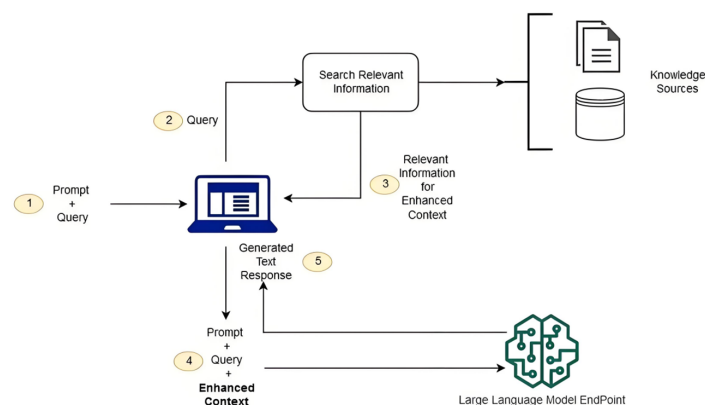


Figure 2.1. High-level RAG workflow from preparation to retrieval and grounded generation.

2.3 Source Preparation, Chunking, and Metadata

The most underestimated part of RAG is source preparation. Teams often treat documents as inert input when in fact documents carry structure, authority, audience, and time. A travel policy, an FAQ written by support, and a draft meeting note may all mention the same subject while possessing radically different evidentiary weight. If they are all ingested into one undifferentiated corpus, the system loses the ability to reason about trust.

Chunking should therefore follow semantic structure whenever possible. Headings, sections, tables, appendices, and frequently updated clauses should influence boundaries. A chunk that contains one complete rule is often more useful than a chunk that contains half of three unrelated ones. At the same time, chunks cannot become too large, or retrieval will be burdened by context dilution and higher token cost. Good chunking is not merely a technical compromise about length; it is a representation choice about what constitutes a meaningful unit of organizational knowledge.

Metadata is what turns a text store into an enterprise knowledge layer. At minimum, documents should preserve source title, document type, owner or department, sensitivity label, effective date, and version or revision marker. Additional metadata such as geography, language, business unit, product line, or audience can become decisive in large deployments. Without metadata, retrieval becomes harder to govern; with metadata, the system can answer not only what text is similar, but what text is permitted, current, and institutionally appropriate.

Design Choice	Why It Matters	Failure If Ignored
Document typing	Distinguishes authoritative policy from informal note	Weak or conflicting evidence enters the answer path
Effective-date metadata	Resolves policy changes over time	Stale passages remain highly retrievable
Sensitivity labels	Supports role-bounded retrieval	Restricted knowledge can leak into general answers
Semantic chunking	Keeps rules whole and retrievable	Relevant evidence is split or diluted

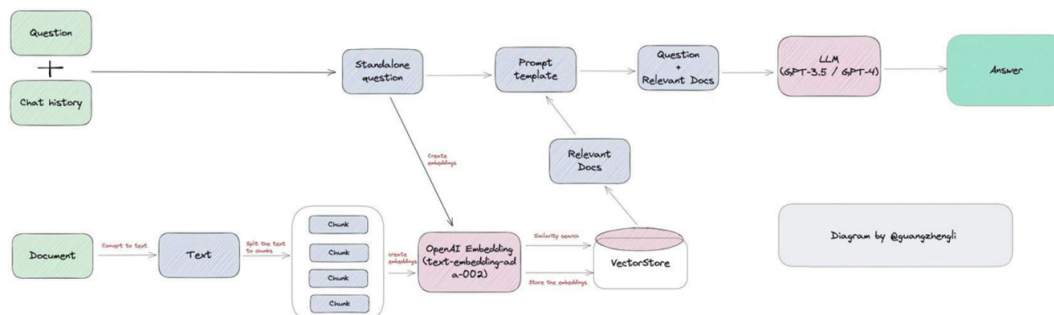


Figure 2.2. From source data to embeddings and similarity-based retrieval in a vector-backed knowledge system.

2.4 Retrieval Strategies beyond “Top-K Vector Search”

Vector similarity search is a strong baseline, but it is not the whole of retrieval engineering. Different question types reward different retrieval behavior. Short factual lookups may benefit from keyword or hybrid retrieval. Ambiguous policy questions may require a reranker. Multi-hop questions may require retrieving from several categories or from both documents and structured sources. The system must therefore be designed as a retrieval strategy engine, not as a single distance formula wrapped in optimism.

Hybrid retrieval is often especially valuable in enterprise settings. Dense retrieval captures semantic similarity and paraphrase well, but exact terms still matter. Policy IDs, form numbers, incident codes, contractual clauses, and product SKUs are frequently brittle under purely semantic matching. A hybrid design that combines vector search with lexical signals or metadata filters can drastically improve precision without sacrificing recall.

Reranking is another crucial layer. The initial retrieval stage can afford to cast a relatively broad net if a second stage scores the candidates more carefully. This two-stage design mirrors many successful information systems: first retrieve generously, then discriminate intelligently. In practice, the reranker may be smaller, cheaper, and more controllable than the final answer model, making it a useful place to invest rigor before the answer prompt ever begins.

2.5 Grounded Generation and Refusal Behavior

Retrieval alone does not produce trustworthy answers. The generation stage must be taught to use evidence rather than merely receive it. A grounded

prompt usually does three things. First, it tells the model to prefer retrieved evidence over prior assumptions. Second, it specifies what to do when evidence is insufficient: ask for clarification, state uncertainty, or refuse to fabricate. Third, it often encourages some visible relationship between answer and source, whether through citation, trace, or explicit phrasing such as “according to the current travel policy.”

Refusal quality deserves more attention than it usually receives. Enterprises often reward helpfulness too strongly and then become surprised when the system invents policy. A mature policy assistant must be able to say that the retrieved materials do not establish an answer. This is not a weakness. It is a mark of institutional honesty. In high-trust environments, a good refusal is frequently safer and more useful than a confident improvisation.

One of the advantages of LinkMind-centered knowledge injection is that refusal policy can be stabilized below several different front ends. Whether the user asks from a web chat, a Lobster console, or a Hermes workflow node, the grounding and refusal behavior can be shaped by one retrieval and generation policy rather than three locally divergent interpretations.

2.6 Hands-On Lab: Building an Internal Policy Assistant

This lab builds a private knowledge assistant for internal policy questions. The scenario is deliberately sober: travel approval, receipt retention, information security responsibilities, and procurement thresholds. Such material is ideal for learning because success depends on precision, freshness, and provenance rather than on conversational charisma. The user should be able to ask a policy question and receive an answer that is not only fluent but grounded in the correct internal source.

The lab architecture has five moving parts. First, a small corpus is prepared with explicit metadata. Second, the corpus is embedded and stored in a vector-capable knowledge layer. Third, LinkMind is configured to expose that knowledge layer to the agent runtime. Fourth, the user queries the assistant with and without retrieval to see the difference. Fifth, the team updates the corpus and verifies that the answer changes without retraining the model. This final step is especially important because it demonstrates the practical advantage of runtime knowledge injection over static model memory.

▷ **Step 1. Curate the corpus as if legal, compliance, and operations will all inspect it later.**

Prepare at least three short but realistic documents: a travel policy, a procurement FAQ, and an information security policy. Assign metadata to every document and preserve it through ingestion. Include title, department, sensitivity, effective date, and document type. If possible, add a revision marker. Even in a small lab, this discipline teaches the reader that ingestion is not simply file upload. It is the moment where enterprise knowledge is translated into runtime evidence.

The documents themselves should be written to create useful contrast. For example, the travel policy may define approval thresholds, the procurement FAQ may define exceptions for emergency purchases, and the security policy may define who owns backup and permissions management. The goal is to create enough variation that retrieval quality can be meaningfully observed.

```
# illustrative metadata envelope
title: Travel Policy 2026
department: Finance
sensitivity: Internal
effective_date: 2026-04-01
source_type: policy
version: 3.1
```

▷ **Step 2. Configure embedding and retrieval through LinkMind.**

The draft material already points to the essential configuration anchors: an embedding backend, a vector store such as Chroma, and a retrieval-enabled request path. Keep the first collection narrow and explicit. A named collection such as 'enterprise-policies' is easier to evaluate than an all-purpose mixed corpus. The purpose of the step is not to memorize vendor syntax; it is to understand that retrieval must be intentionally wired into the runtime rather than assumed as a side effect.

If the organization already has a separate RAG platform and only needs a shared vectorization service, this is where the Embedding Server pattern becomes relevant. The educational insight is that vectorization and retrieval do not always need to be owned by the same operational team. What matters is that their boundary remains explicit and that agent clients see a consistent

knowledge interface above them.

```
# illustrative skeleton only
functions:
  embedding:
    backends:
      - name: primary-embedding
        enable: true

rag:
  enabled: true
  store: chroma
  collection: enterprise-policies
  categories:
    - policies
```

▷ Step 3. Run paired prompts with retrieval off and on.

- Ask: “Who approves overseas travel above USD 2,000?” with retrieval disabled or without a knowledge category.
- Ask the same question again with the policy corpus explicitly enabled.
- Ask: “How long must expense receipts be retained?” and verify that the answer attaches to the correct policy source.
- Ask a question outside the corpus, such as a product support question, and inspect whether the assistant refuses appropriately.

The comparison usually reveals a dramatic change in answer posture. Without retrieval, the model may sound generally helpful but vague. With retrieval, it should become both more concrete and more bounded, often citing or reflecting the specific wording of the document. If the answer becomes more verbose but not more authoritative, the problem is probably upstream in retrieval or prompt framing.

▷ Step 4. Test freshness by changing the rule.

Update one policy value, such as the overseas travel approval threshold, and ingest the changed document or chunk. Then repeat the question. This is the pedagogical moment where RAG becomes tangible. The answer should change because the corpus changed, not because the model was retrained.

If the answer does not change, the team has learned something valuable: either the wrong version is being retrieved, the effective-date metadata is insufficient, or the retrieval cache is too stale.

In an enterprise deployment this freshness test is not optional. Many highly publicized AI failures are not failures of raw intelligence; they are failures of update discipline. A policy assistant that cannot change with the institution is little more than a persuasive archive.

▷ **Step 5. Inspect trace, provenance, and refusal behavior together.**

Ask not only whether the answer sounds right, but whether the runtime can expose enough evidence for a reviewer to trust it. A useful trace often shows retrieved chunks, document identifiers, or source labels. A useful refusal path explains that the answer is not established by the available corpus. The combination of provenance and refusal is what separates grounded knowledge systems from polished guesswork.

2.7 Metrics and Evaluation for Private Knowledge Systems

Metric	What It Captures	Why It Matters
Grounding accuracy	Whether answers align with retrieved evidence	Prevents fluent but unsupported responses
Retrieval relevance	Whether the right chunks are surfaced	Localizes failure to the retrieval stage
Freshness latency	How quickly document updates affect answers	Measures operational agility
Refusal quality	Whether the system declines unsupported answers gracefully	Protects trust in high-stakes use cases
Trace completeness	Whether evidence can be inspected after the fact	Supports audit and debugging

Evaluation should mix automatic checks and human review. Automatic tests are useful for repeated policy questions with stable expected answers. Human review remains essential for ambiguity, edge cases, and the interpreta-

tion of conflicts between sources. Over time, the best knowledge teams build a benchmark set of canonical internal questions and treat regression against that set as a release risk.

2.8 Common Anti-Patterns

- Uploading documents without metadata and then wondering why governance is weak.
- Treating chunk size as a purely technical parameter instead of a semantic design choice.
- Assuming vector similarity alone is enough for every question type.
- Rewarding helpfulness so strongly that the system stops refusing unsupported questions.
- Believing that retrieval eliminates hallucination automatically without careful answer framing and evaluation.

2.9 Multi-Source Knowledge and Structured Data

Document retrieval is often the first form of private knowledge injection because it is accessible and visually intuitive. Yet enterprise knowledge rarely lives in documents alone. Some of it lives in databases, ticket systems, contract tables, product catalogs, or HR systems. A mature knowledge architecture therefore combines narrative sources and structured sources rather than pretending that every question is best answered by paragraph retrieval.

The engineering challenge is not simply to connect more sources. It is to preserve source identity and query discipline. A question about who approves overseas travel might be answered from a policy document. A question about whether a particular employee has already completed a required training may depend on a live record system. A question about the current owner of a device belongs in an asset database, not in a PDF. If the system routes all such questions into one undifferentiated retrieval path, it will often surface something textually similar rather than operationally authoritative.

This is one reason LinkMind's broader positioning around connecting MySQL, Elasticsearch, Chroma, and other backends is pedagogically valuable. The platform reminds the reader that "private knowledge" is not synonymous with "private documents." It is an umbrella for many evidence channels whose

semantics differ and whose governance burdens differ accordingly.

Multi-source knowledge injection becomes especially powerful when the answer path can merge evidence types without confusing them. A strong policy assistant might retrieve the relevant approval rule from a document, confirm the employee's department from a directory, and then produce a response that distinguishes normative rule from live record. Such answers feel natural to users because they mirror how institutions actually reason.

2.10 Provenance, Citation, and Evidence Presentation

Provenance is not merely a technical convenience for debugging. It is one of the cultural bridges by which organizations learn to trust machine assistance. When a system cites a policy section, a document title, an effective date, or a source category, it tells the user that the answer has a basis outside the model's style. This does not make the answer automatically correct, but it makes the answer disputable in a productive way. Users can challenge the evidence rather than arguing about the mysterious interior of the model.

Citation practices should fit the use case. Some systems need inline citations, especially in regulated or advisory settings. Others need a trace panel or expandable evidence list so that the main answer remains readable. The key is consistency. If sometimes the system grounds visibly and sometimes it does not, users begin to form unreliable intuitions about when they may trust it. Consistency is one of the hidden benefits of centralizing knowledge behavior in a layer such as LinkMind. Several front ends can present evidence differently while still sharing one provenance contract underneath.

Evidence presentation also influences model behavior. If the system is instructed to answer in terms of retrieved material and to name the source context when appropriate, it tends to remain more disciplined. If it is rewarded only for sounding complete, it may drift toward synthesis untethered from evidence. In other words, provenance is both a user-facing trust device and a model-facing grounding device.

In high-importance domains, provenance should support after-the-fact review as well as immediate user interpretation. Operators may need to know not only which chunk was retrieved, but which version of the document, which category filter, which embedding model, and which reranking path contributed to the answer. That level of evidence is rarely needed by end users, but it is

invaluable for platform maturity.

2.11 Knowledge Governance over Time

Knowledge systems degrade quietly if ownership is unclear. Someone must own each corpus, define its refresh cycle, approve revisions, and retire obsolete material. Without this governance, retrieval becomes an elegant path into confusion. The model may answer from the wrong version of a policy not because retrieval failed technically, but because no one ever declared which version should remain authoritative.

Governance also includes deletion and revocation. Enterprises sometimes focus on ingestion pipelines while ignoring offboarding. Yet records expire, policies are superseded, and sensitive files lose their right to be retrievable for certain audiences. A robust knowledge pipeline must therefore support not only adding more evidence but also withdrawing or narrowing evidence. This is especially important in environments with employee movement, regulatory change, or mergers of business lines.

LinkMind is not itself a document management system, and it should not be mistaken for one. Its relevance is that it can sit where governance policies become runtime behavior. A category can be restricted. A corpus can be rerouted. A source can be excluded. This makes the platform a lever through which knowledge governance affects live answers rather than remaining a separate administrative ideal.

The most effective knowledge programs often institutionalize a small review rhythm. Monthly checks on top questions, freshness gaps, failed refusals, and disputed answers can reveal whether the corpus is aging badly or whether the retrieval design is no longer aligned with actual business demand.

2.12 When Retrieval Is Not Enough

Retrieval is powerful, but it does not solve every domain problem elegantly. Some use cases require stable response style, domain-specific terminology, or specialized reasoning patterns that retrieval alone may not provide. In such cases, teams sometimes consider fine-tuning, supervised examples, or hybrid approaches. The correct textbook stance is neither to worship fine-tuning nor to reject it reflexively. The better question is which part of the problem is best addressed by which mechanism.

Retrieval is usually the right answer for freshness and evidence. Fine-tuning may be useful for style, structured extraction behavior, or domain phrasing. Workflow constraints may be useful for high-risk reasoning paths. Capability calls may be useful when the real answer lives in structured systems. Once the problem is decomposed this way, architectural choices become more sober. The team is no longer asking “should we use RAG or fine-tuning?” as though the system must belong to one tribe. It is asking which components deserve which treatment.

A healthy enterprise stack may therefore combine several methods. LinkMind can route the request, apply retrieval, expose capabilities, and then send the final answer stage through a model that has been adapted for the organization’s preferred style. This layered perspective is one of the reasons middleware-centered design scales conceptually. It resists false dichotomies by giving each concern a place.

The practical caution is that each additional mechanism increases evaluation burden. Hybrid systems can become genuinely excellent, but only if the team maintains discipline about what each layer is supposed to contribute and how regression will be detected when one layer changes.

Chapter Summary

- Private knowledge injection turns enterprise truth into a runtime evidence problem rather than a retraining cycle.
- RAG quality depends on ingestion, metadata, chunking, retrieval, reranking, and grounded generation working together.
- LinkMind is valuable because it centralizes the knowledge boundary below many possible agent clients.
- Freshness and refusal are as important as relevance in serious knowledge systems.
- A policy assistant becomes trustworthy when its answers can be updated, inspected, and challenged through evidence.

Review Questions

- Why should enterprises distinguish operational truth from general model knowledge?

- How does metadata change what a retrieval system can govern?
- What kinds of questions are poorly served by vector similarity alone?
- Why is a good refusal often more valuable than a plausible answer?
- How would you test whether a knowledge assistant is fresh rather than merely fluent?

Further Study: Conflicting Truth and Source Priority

Enterprises often speak of a single source of truth, yet working systems frequently encounter several sources of partial truth. A corporate policy may say one thing, a regional playbook another, and an operations FAQ a third. A retrieval system that ignores these layers will often surface whichever passage is semantically closest, not whichever passage is institutionally decisive. This is one reason that knowledge engineering cannot remain a purely technical problem. It is partly a problem of institutional authority expressed through metadata and retrieval policy.

Source priority is therefore an important design concept. Official policies may outrank FAQs. Current effective versions may outrank historical ones. Regional documents may outrank corporate ones only within clearly specified jurisdictional conditions. Once such priorities are made explicit, the assistant can behave less like a clever reader and more like an informed participant in the institution's own logic. This change is subtle but profound. It moves the system from superficial semantic matching toward operational fidelity.

Even with good priority rules, some conflicts should remain visible rather than hidden. There are situations in which the right answer is that the institution itself is in transition or disagreement. In these moments, a textbook-quality assistant should escalate conflict clearly, identify the competing sources, and point to the owner responsible for resolution. The refusal to flatten conflict into a false certainty is one of the strongest signals that the system respects enterprise reality.

A centralized layer such as LinkMind becomes strategically useful here because source-priority behavior can be encoded once and applied broadly. Without such centralization, different teams may quietly teach their local assistants to resolve the same conflict differently, producing inconsistency that users interpret as unreliability.

Further Study: Review Programs and Benchmark Stewardship

A knowledge assistant matures when the organization builds not only a corpus, but a review program around that corpus. Benchmark stewardship means selecting representative questions, attaching expected evidence, replaying the set after changes, and recording disagreement. This work is rarely glamorous, but it is one of the strongest protections against slow quality drift. As corpora grow, as chunking strategies evolve, and as prompts change, the benchmark becomes the memory of what the system once knew how to do.

Benchmark stewardship also creates a bridge between platform teams and domain experts. Platform teams can automate replay, trace collection, and regression alerts. Domain experts can judge whether the retrieved evidence and the answer interpretation are institutionally correct. Neither group can perform the task adequately alone. The benchmark is therefore both a technical artifact and a collaboration artifact.

Over time, strong teams develop several classes of benchmark items. Some test straightforward grounding. Some test freshness after document updates. Some test refusals on unsupported questions. Some test authority conflicts. Some test whether role-based corpus boundaries are respected. Together these items create a far richer picture of system health than anecdotal demonstrations ever can.

LinkMind strengthens such a program because one retrieval layer can serve many front ends. The same benchmark can reveal whether a regression affects the whole platform rather than only one interface. That leverage is precisely what makes middleware-centered design so valuable in enterprise knowledge work.